

HRMS

Foundation / Reference Data

Country, State, AddressType, Relationship, Gender, MaritalStatus, IdentifierType, DocumentType, Skill, Certification, Language, EmploymentType and EmploymentStatus

Technical implementation reference

1. Scope and Purpose

This document defines the technical implementation and operating contract for the HRMS Foundation/reference-data layer. Foundation data is intentionally centralized so Employee and other modules do not duplicate controlled values.

2. Architecture

HRMS follows a modular-monolith architecture. The API project is the composition root. Business use cases are implemented with MediatR commands/queries and handlers. Domain entities contain domain concepts and invariants. Infrastructure contains persistence, security and external implementation details. Shared BuildingBlocks provide abstractions such as IUserContext, IReadRepository and IWriteRepository. SQL Server is the persistence store and EF Core migrations manage schema evolution.

Application handlers should depend on abstractions rather than ApplicationDbContext. Read operations use IReadRepository; write operations use IWriteRepository. Infrastructure supplies the repository implementations. This keeps business logic testable and prevents API controllers from becoming business-rule containers.

Module responsibility

Foundation owns reusable reference data and its lifecycle. Employee consumes Foundation IDs rather than storing free-text values where a controlled master exists. API controllers should remain thin and delegate to MediatR handlers.

3. Implemented Foundation Masters

Logical entity	SQL table	Purpose / key relationships
Country	Countries	Country master used by address and geographic data
State	States	State/province master; normally references Country
AddressType	AddressTypes	Classifies employee addresses such as current/permanent
Relationship	Relationships	Relationship master for emergency contacts/dependents/nominees
Gender	Genders	Controlled gender reference data
MaritalStatus	MaritalStatuses	Controlled marital-status reference data
IdentifierType	IdentifierTypes	Defines government/identity identifier categories
DocumentType	DocumentTypes	Defines employee document categories
Skill	Skills	Reusable employee skill master
Certification	Certifications	Certification master used by employee certifications
Language	Languages	Language master used by employee language records
EmploymentType	EmploymentTypes	Employment engagement/type classification
EmploymentStatus	EmploymentStatuses	Current employment lifecycle/status classification

4. SQL Table Information

The following is the logical SQL Server table catalog. Table names shown here are the intended/implemented plural SQL naming convention; exact physical names should be verified against the current EF Core model before writing hand-maintained SQL.

Field	Typical SQL type	Purpose
Id	uniqueidentifier	Primary key

Field	Typical SQL type	Purpose
CreatedBy	uniqueidentifier NULL	User that created the row
CreatedOn	datetimeoffset	Creation timestamp
ModifiedBy	uniqueidentifier NULL	Last modifying user
ModifiedOn	datetimeoffset NULL	Last modification timestamp
DeletedBy	uniqueidentifier NULL	User that soft-deleted the row
DeletedOn	datetimeoffset NULL	Soft-delete timestamp
IsDeleted	bit	Soft-delete flag

Common master-data columns

Foundation master entities generally require an identifier, display/name value, active-state control and the project's standard audit/soft-delete fields. Where an entity has a parent reference, the foreign key should be indexed and enforced by the EF model.

Relationship examples

State → Country is a parent-child relationship. Employee Address → AddressType/Country/State links employee transactional data to Foundation. Employee Government Identifier → IdentifierType. Employee Document → DocumentType. Employee Skill/Certification/Language → the corresponding master.

5. API Contract

Operation	HTTP	Endpoint pattern	Purpose
List	GET	/api/<Resource>	Return active/non-deleted master records
Get by ID	GET	/api/<Resource>/{id}	Return one master record
Create	POST	/api/<Resource>	Create a master record
Update	PUT	/api/<Resource>/{id}	Modify an existing master record
Deactivate	POST	/api/<Resource>/{id}/deactivate	Deactivate a record without physical deletion

Create example

```
POST /api/Countries
Content-Type: application/json

{
  "name": "India"
}
```

Expected response

```
201 Created

{
  "id": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"
}
```

Get example

```
GET /api/Countries/{id}

200 OK
{
  "id": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "name": "India",
  "isActive": true
}
```

6. Business Rules

1. Names/descriptions of reference values must satisfy command validation and should not be blank.
2. A reference record should not be physically deleted when it may already be referenced by employee data; deactivation/soft deletion is preferred.
3. Duplicate business names/codes should be prevented where the domain defines them as unique.
4. Foreign-key references must point to existing active/non-deleted records unless a historical use case explicitly permits inactive values.
5. Updates must validate route ID versus command ID when both are supplied.
6. Controllers must not contain persistence queries or business-rule decisions beyond HTTP concerns.

7. Developer Change Guide

To add a new Foundation master: create the Domain entity, configure EF mapping/indexes, add Application DTOs/commands/queries/validators/handlers, expose the controller endpoint, register required dependencies, add/update the migration, and test Create/Get/Update/Deactivate plus invalid and duplicate scenarios.

Migration discipline

```
dotnet ef migrations add \  
--project "..\src\HRMS.Persistence" \  
--startup-project "..\src\HRMS.API"
```

Review generated SQL/migration operations before applying them to an existing database. Avoid destructive table recreation for reference data that is already populated.

8. Recommended Improvements

Centralize common master-data validation, standardize error responses, add unique indexes for business keys, document seed data separately from runtime CRUD, and add integration tests covering FK/deactivation behavior.